

Microservices application

1 Chosen domain

The chosen domain for this project is IoT Security Monitoring. The system is designed to monitor and manage security-related activity generated by Internet of Things (IoT) devices such as cameras, sensors, access-control devices and other connected equipment.

IoT environments can contain a large number of connected devices, which increases the potential attack surface of an organisation. The application provides functionality for registering and managing IoT devices, recording security events and authenticating users.

The domain was selected because it is well suited to a microservices architecture. Device management, authentication and security-event processing represent separate business capabilities that can be developed, deployed and scaled independently.

The system also provides a foundation for integration with a future Security Operations Centre (SOC). Logs and security events can be collected and analysed using technologies such as Elasticsearch, Logstash and Kibana, while the service-oriented design allows additional security-monitoring components to be introduced later.

2 Microservices architecture

The application follows a microservices architecture, where the system is divided into several independently deployable services. Each service is responsible for a specific business capability rather than implementing all functionality in a single application.

The main components of the system are:

- Client application
- Eureka Service Registry
- API Gateway
- Device Service
- Event Service
- Authentication Service
- PostgreSQL databases
- Elasticsearch

- Logstash
- Kibana

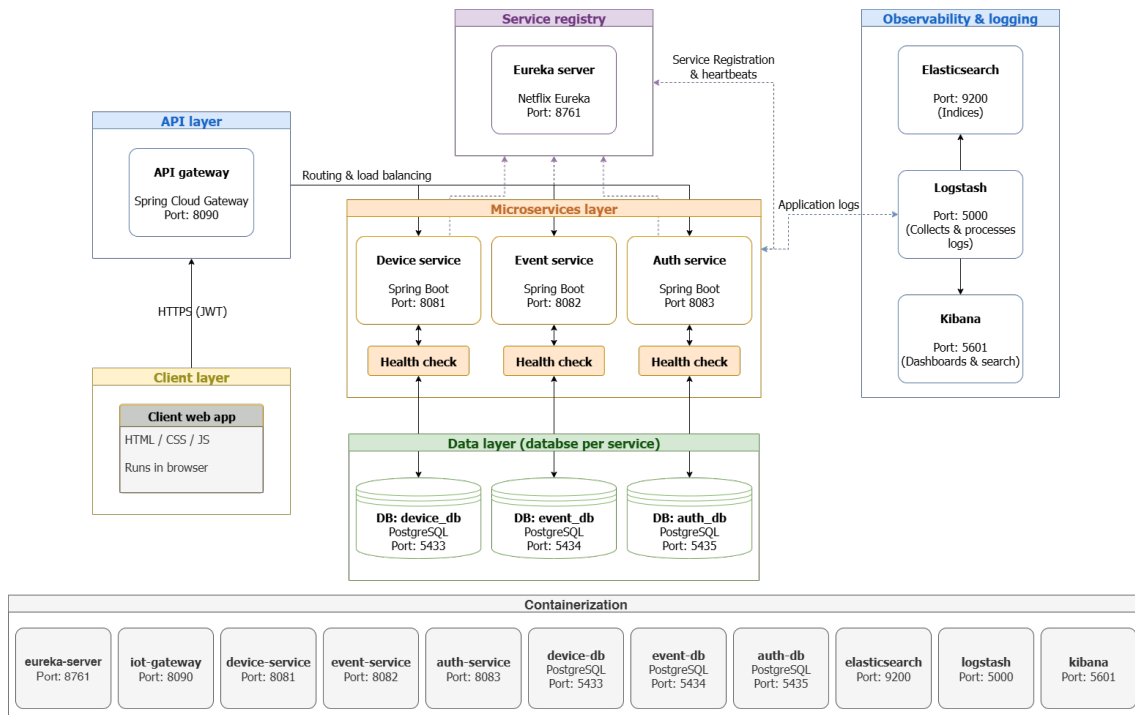


Figure 1: System architecture

The client application communicates with the system through the API Gateway rather than communicating directly with individual microservices. The gateway determines which service should receive each request.

The authentication service manages user authentication and JWT-based security. The device service manages IoT device information, while the event service manages security events.

Each business service has its own database, following the database-per-service microservices pattern:

- Authentication Service → auth_db
- Device Service → device_db
- Event Service → event_db

The architecture also separates application functionality from monitoring infrastructure. Elasticsearch, Logstash and Kibana provide a centralised logging and monitoring capability without making the business services responsible for directly implementing log analysis.

3 Role of each service

3.1 Client application

The client application provides the user interface for interacting with the IoT Security Monitoring System. It is implemented using HTML, CSS and JavaScript.

3.2 Eureka service registry

The Eureka server provides service discovery for the microservices environment. It runs on port **8761** and maintains information about registered service instances. Instead of requiring services to rely entirely on hard-coded internal addresses, services can register themselves with Eureka and discover other services through the registry.

3.3 API gateway

The API gateway acts as the single entry point into the backend system. It receives requests from the client and routes them to the appropriate microservice. The gateway runs on port **8090**.

3.4 Device service

The device service manages information relating to IoT devices. The device service runs on port **8081** and uses the `device_db` PostgreSQL database. Keeping device functionality in a separate service allows device management to evolve independently from authentication and security-event processing.

3.5 Event service

The event service manages security events generated by or associated with IoT devices. The event service runs on port **8082** and uses the `event_db` PostgreSQL database. Examples of supported event types include: `UNAUTHORIZED_ACCESS`, `FAILED_LOGIN`, and `BRUTE_FORCE`.

3.6 Authentication service

The authentication service is responsible for user authentication and authorisation-related functionality. The service runs on port **8083** and uses the `auth_db` PostgreSQL

database. After a successful login, the service provides a JWT that the client can use when accessing protected endpoints.

3.7 Logging and monitoring services

The system also includes:

- Logstash on port **5000**
- Elasticsearch on port **9200**
- Kibana on port **5601**

Logstash receives and processes application logs, Elasticsearch stores and indexes log information, and Kibana provides a visual interface for viewing and analysing logs.

4 Endpoint design

The API follows a RESTful endpoint design. Resources are represented using nouns, while HTTP methods indicate the operation being performed. The API Gateway exposes the public API using the `/api` prefix.

Authentication endpoints

The Authentication Service provides endpoints under: `/api/auth/**`

Operations include:

Method	Endpoint	Purpose
POST	<code>/api/auth/register</code>	Register user
POST	<code>/api/auth/login</code>	Authenticate user
GET	<code>/api/auth/me</code>	Get current authenticated user

Device endpoints

The Device Service uses: `/api/devices/**`

Operations include:

Method	Endpoint	Purpose
GET	/api/devices	Get all devices
GET	/api/devices/id	Get device by database ID
GET	/api/devices/device-id/deviceId	Get device by ID
POST	/api/devices	Register a device
PUT	/api/devices/id	Update a device
DELETE	/api/devices/id	Delete a device

Event endpoints

The Event Service uses: /api/events/**

Operations include:

Method	Endpoint	Purpose
GET	/api/events	Get all security events
GET	/api/events/id	Get security event by event ID
GET	/api/events/status/status	Get security events by status
GET	/api/events/device/deviceId	Get security events by device
POST	/api/events	Create security event
PUT	/api/events/eventId	Update security event
PATCH	/api/events/eventId/status	Update security event status
DELETE	/api/events/eventId	Delete security event

5 Selected microservices patterns

5.1 Database per service

Each business service owns a separate database. The architecture uses:

- device_db
- event_db
- auth_db

This prevents one service from directly depending on another service's database schema. It also allows individual services to change their persistence implementation without affecting the other services.

5.2 API gateway

The API gateway provides a single entry point for the client. Instead of exposing every microservice directly to the client, requests are sent to port **8090**, and the gateway routes them internally. This simplifies the client architecture and provides a central location for security, routing and other cross-cutting concerns.

5.3 Centralised logging

Logs from the distributed services are collected into a central logging pipeline. Logstash processes the logs and sends them to Elasticsearch, while Kibana provides visualisation and analysis. This is important because debugging a distributed system is more difficult when logs are stored independently on each service.

5.4 Security and access tokens

The authentication mechanism uses JWT tokens. Once authenticated, the client sends the JWT with subsequent protected requests. This avoids requiring the backend services to maintain a traditional server-side session for every client.

5.5 Independent service deployment

Each microservice is packaged and deployed independently in Docker. This allows a service to be modified, rebuilt and redeployed without requiring the entire application to be redeployed. This is one of the main advantages of the microservices approach.

5.6 Health check API

The services expose health information through Spring Boot Actuator endpoints. Health checks can be used to determine whether a service is running correctly and can later be integrated into container orchestration and monitoring infrastructure.

5.7 Circuit breaker

A circuit breaker has been implemented between the device service and event service to improve the resilience of the system. When the event service communicates with the device service and repeated failures occur, the circuit breaker can temporarily stop further requests from being sent to the unavailable service. This prevents cascading failures and allows the event service to continue operating where possible. After a

defined recovery period, the circuit breaker allows requests to be tested again and can automatically return to normal operation when the device service becomes available. This improves fault tolerance and ensures that a temporary failure in one microservice does not unnecessarily bring down other parts of the application.

6 API gateway configuration

The API gateway is implemented using Spring Cloud Gateway and operates on port **8090**. The gateway contains routes that map public API paths to the corresponding backend services. The main routing configuration is:

- `/api/devices/**` → Device Service
- `/api/events/**` → Event Service
- `/api/auth/**` → Authentication Service

The gateway therefore hides the internal service ports from the client.

For example, the client does not need to directly access: `http://localhost:8081`. Instead, it sends a request to: `http://localhost:8090/api/devices`. The gateway then forwards the request to the device service.

This design provides several benefits. Internal service addresses are not exposed to the client, routing rules can be changed centrally, and security-related processing can be placed at the gateway layer. The gateway can also act as an important future security enforcement point. For example, additional JWT validation, rate limiting, request filtering and auditing can be introduced without requiring every client to implement these mechanisms independently.

7 Service registry and discovery

The system uses Netflix Eureka as its service registry. The Eureka server operates on port **8761**. When a microservice starts, it registers itself with Eureka. Other components can then discover available service instances through the registry. The main registered services include:

- API Gateway
- Authentication Service
- Device Service
- Event Service

The registry provides a layer of abstraction between services and their physical deployment locations. This is particularly valuable when moving from local development to containerised deployment. For example, a service may run on a different container address after deployment. Service discovery reduces the need to manually update every component with the new address.

Eureka also provides a foundation for scaling. If multiple instances of a service are deployed, the registry can maintain information about the available instances.

8 Deployment approach

The application uses Docker containerisation for deployment. The architecture separates the different application components into containers, allowing the microservices and supporting infrastructure to run as independent components. The main deployment components include:

- Eureka Server
- API Gateway
- Device Service
- Event Service
- Authentication Service
- PostgreSQL
- Elasticsearch
- Logstash
- Kibana

Docker Compose can be used to coordinate the containers during development and testing. The main service ports are:

Component	Port
Eureka Server	8761
API Gateway	8090
Device Service	8081
Event Service	8082
Authentication Service	8083
PostgreSQL	5432
Elasticsearch	9200
Logstash	5000
Kibana	5601

Containerisation provides consistency between development and deployment environments. Dependencies such as PostgreSQL and Elasticsearch can be started alongside the application rather than requiring every component to be manually installed and configured on the host machine.

9 Preparation for future SOC

The architecture has been designed so that it can be extended into a more comprehensive Security Operations Centre environment. A future SOC would require the ability to collect security information from many sources, detect suspicious behaviour, correlate events and provide security personnel with actionable information. The current application provides several foundations for this.

9.1 Centralised security events

Security events are stored by the event service using a structured event model. Events include information such as `event_id`, `device_id`, `event_type`, and `severity`. Because security events are structured, a future SOC component can consume and analyse them.

9.2 Centralised logging

The use of Logstash, Elasticsearch and Kibana provides a foundation for centralised security monitoring. Application logs from the microservices can be collected, indexed and visualised. A future SOC can use these logs to identify patterns such as repeated authentication failures or unusual device activity.

9.3 Security and event classification

Security events use predefined event types and severity levels. This allows future detection systems to distinguish between different levels of risk. For example, repeated `FAILED_LOGIN` events could be correlated into a possible brute-force attack, while `MALWARE_DETECTED` could generate a higher-priority security alert.

9.4 Authentication and JWT

The use of JWT-based authentication provides a foundation for securing access to the system. A future SOC can monitor authentication-related events, including failed login attempts and suspicious access patterns.

9.5 Gateway as security control point

The API Gateway provides a central location where additional security controls can be introduced. Future functionality could include:

- Rate limiting.
- Request filtering.
- IP-based controls.
- Enhanced JWT validation.
- API auditing.
- Threat detection.
- Access logging.

10 Conclusion

The IoT Security Monitoring System uses a microservices architecture to separate authentication, device management and security-event management into independent services. The API gateway provides a single entry point for clients, while Eureka provides service discovery. The database-per-service approach provides data isolation, and Docker provides a consistent deployment model.

The system also incorporates centralised logging through Logstash, Elasticsearch and Kibana. This is particularly important for a security-focused application because monitoring and analysing activity across distributed services is a fundamental requirement for security operations.

Most importantly, the architecture has been designed with future extensibility in mind. Structured security events, centralised logs, JWT authentication, gateway-based access and independent services provide a strong foundation for adding security analytics, alerting, event correlation and eventual SOC integration.

The architecture diagram complements these explanations by showing how the client, gateway, service registry, microservices, databases and monitoring infrastructure interact as a complete distributed system.